

Table of contents

1	Casting Rays	2
1.1	Reverse Ray Tracing	2
1.2	The Ray Equation	3
1.3	Intersection: The Sphere	3
	1.3.1 The Math	3
	1.3.2 Implementation	4
1.4	Ray Marching Visualization	5
1.5	Injection	6

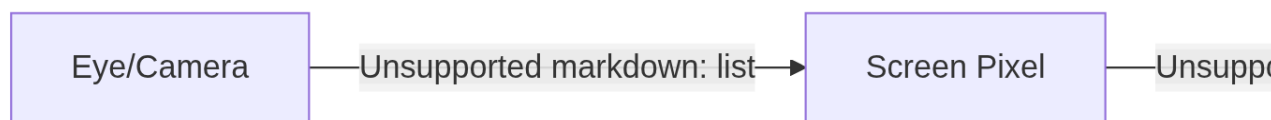
Chapter 1

Casting Rays

With our grid (Canvas) and maths (Vectors) established, simulation begins. The first step is locating the light.

1.1 Reverse Ray Tracing

In reality, light travels from a bulb to your eye. In computer graphics, we work in reverse. We shoot a ray from the “Camera” (your eye) through every screen pixel to see what it strikes.



Ray tracing flow: from eye through screen to scene

💡 Why Reverse?

Forward ray tracing (light $\rightarrow$ eye) is inefficient because most rays miss the camera. Reverse tracing (eye $\rightarrow$ scene) ensures every ray contributes to the image.

1.2 The Ray Equation

A ray is defined mathematically as:

$$\mathbf{P}(t) = \mathbf{O} + t\mathbf{D}$$

Where: - $\mathbf{O}$ = **Origin** (ray start position) - $\mathbf{D}$ = **Direction** (normalised vector)
- t = **Distance** along the ray - $\mathbf{P}(t)$ = **Point** on the ray at distance t

```
// Ray representation in code
const ray = {
  origin: {x: 0, y: 0},
  direction: {x: 1, y: 0} // Must be normalised!
};

// Point at distance t
function rayPoint(ray, t) {
  return {
    x: ray.origin.x + t * ray.direction.x,
    y: ray.origin.y + t * ray.direction.y
  };
}
```

1.3 Intersection: The Sphere

Our scene comprises spheres. To check if a ray hits a sphere, we use algebra. A sphere is defined as all points at a distance r from a centre point C .

1.3.1 The Math

A sphere centred at $\mathbf{C}$ with radius r satisfies:

$$|\mathbf{P} - \mathbf{C}|^2 = r^2$$

Substituting the ray equation $\mathbf{P}(t) = \mathbf{O} + t\mathbf{D}$:

$$|\mathbf{O} + t\mathbf{D} - \mathbf{C}|^2 = r^2$$

This becomes a quadratic equation:

$$at^2 + bt + c = 0$$

Where: - $a = \mathbf{D} \cdot \mathbf{D}$ (usually 1 if direction is normalised) - $b = 2\mathbf{D} \cdot (\mathbf{O} - \mathbf{C})$ - $c = |\mathbf{O} - \mathbf{C}|^2 - r^2$

The **discriminant** $\Delta = b^2 - 4ac$ determines if we hit:

Discriminant	Result
$\Delta < 0$	No intersection (ray misses sphere)
$\Delta = 0$	Tangent (ray grazes sphere)
$\Delta > 0$	Two intersections (ray enters and exits)

1.3.2 Implementation

This is implemented in `Scene.trace` within `src/engine.js`:

```
/* src/engine.js */
trace: function(ro, rd) {
  // ...
  const oc = {x: ro.x - sphere.x, y: ro.y - sphere.y};
  const b = 2 * (rd.x * oc.x + rd.y * oc.y);
  const c = (oc.x * oc.x + oc.y * oc.y) - r * r;

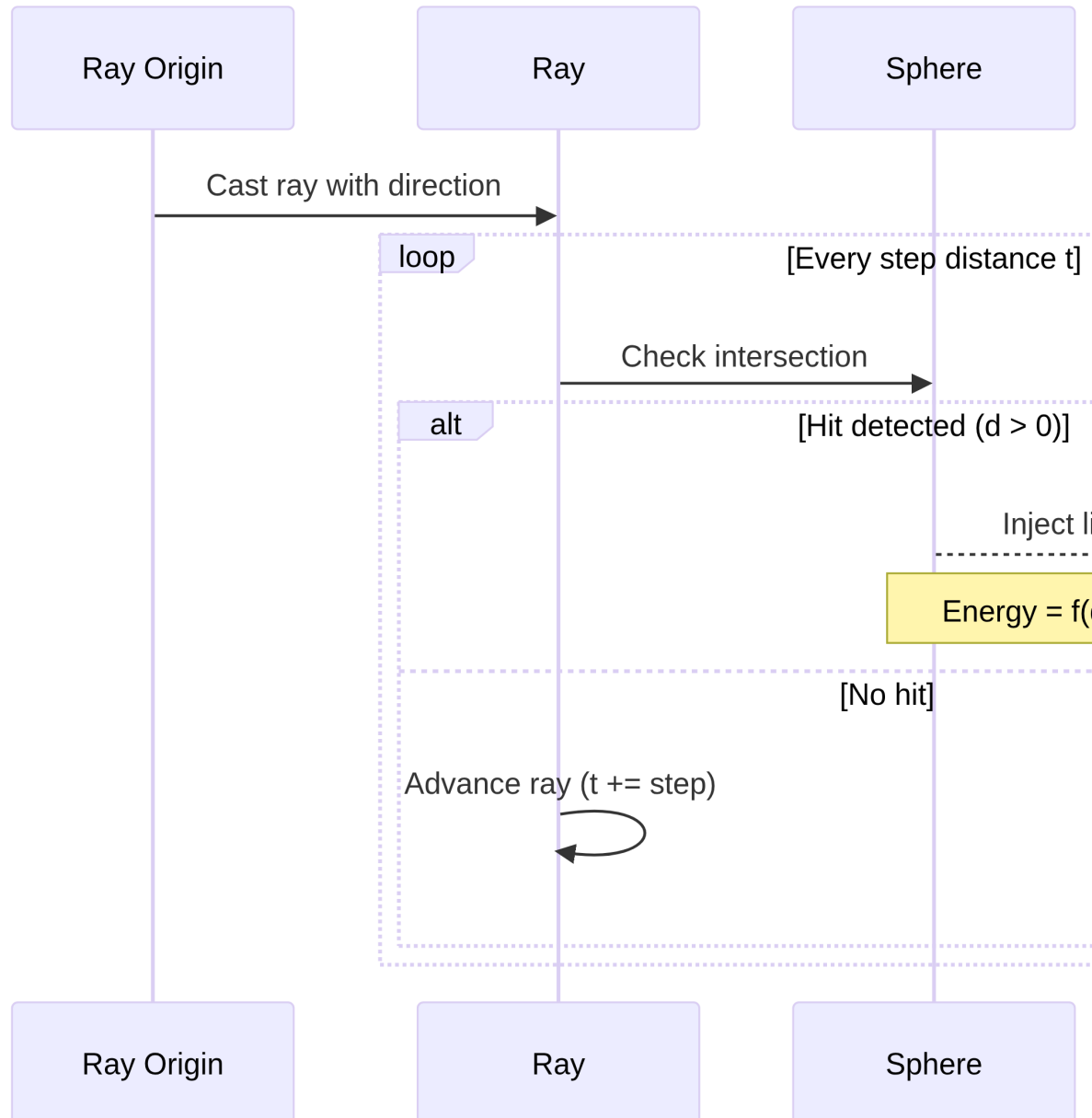
  const d = b * b - c;
  if (d > 0) {
    const t = (-b - Math.sqrt(d)) / 2;
    // We hit the sphere at distance t!
  }
}
```

- ① Calculates coefficient $b = 2\mathbf{D} \cdot (\mathbf{O} - \mathbf{C})$
- ② Calculates coefficient $c = |\mathbf{O} - \mathbf{C}|^2 - r^2$
- ③ Computes discriminant $\Delta = b^2 - 4ac$ (simplified since $a = 1$)
- ④ Positive discriminant means intersection
- ⑤ Solves for t using quadratic formula (takes nearest hit)

💡 API Reference

See the complete implementation in [API Reference](#) → `Scene.trace`.

1.4 Ray Marching Visualization



Step-by-step ray marching through a scene

1.5 Injection

When a ray strikes a light source, we “inject” that energy into our fluid grid (`State.lattice`). This initiates our fluid simulation.

```
// Simplified light injection
function injectLight(hitPoint, energy) {
  const pixelX = Math.floor(hitPoint.x);
  const pixelY = Math.floor(hitPoint.y);
  const index = pixelY * WIDTH + pixelX;

  lattice[index * STRIDE + INTENSITY] += energy;
}
```

The injected energy then propagates via fluid dynamics (see [Chapter 6: Fluids](#)).

i Performance Considerations

- **Ray count:** More rays = better quality but slower
- **Step size:** Smaller steps = more accurate but more expensive
- **Early termination:** Stop after first hit to save computation

[→ Next: Fluid Simulation](#) | [← Previous: Mathematics](#)